

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ  
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное  
бюджетное**

**образовательное учреждение высшего образования  
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

**Лабораторная работа №5**

**Строки и регулярные выражения**

**по дисциплине**

**«Информационные технологии и программирование»**

Выполнил: студент гр. БВТ2403  
Толибов Р.Ш.

Руководитель:

Москва, 2025 г.

**Цель работы:** изучить особенности работы со строками в языке программирования Java, принципы их неизменяемости и интернирования, а также освоить применение регулярных выражений для поиска, проверки и обработки текстовой информации.

**Ход работы:**

Для оптимизации работы со строками JVM использует механизм интернирования, при котором одинаковые строковые литералы хранятся в специальном пуле строк. Это позволяет уменьшить количество дублирующихся объектов в памяти.

Строки в Java хранятся в кодировке UTF-16. Это означает, что некоторые символы Unicode (например, эмодзи) могут занимать более одной 16-битной кодовой единицы. В связи с этим различают длину строки в code points и code units.

Регулярные выражения представляют собой мощный инструмент для работы с текстом. В Java они реализованы с помощью классов Pattern и Matcher, которые позволяют выполнять поиск, проверку и замену фрагментов строк по заданному шаблону.

Задание 1. Поиск всех чисел в тексте.

Была реализована программа, осуществляющая поиск всех числовых значений в заданной строке. Для этого использовалось регулярное выражение, описывающее формат числа. Найденные совпадения выводились на экран с помощью объекта Matcher.

```
import java.util.regex.*;

public class NumberFinder {
    public static void main(String[] args) {
        try {
            String text = "БВТ2403, Информатика и вычислительная техника (09.03.01)";
            Pattern pattern = Pattern.compile("\\d+(\\.\\d+\\.\\d+)?");
            Matcher matcher = pattern.matcher(text);
            while (matcher.find()) {
                System.out.println(matcher.group());
            }
        } catch (Exception e) {
            System.out.println("Произошла ошибка! " + e.getMessage());
        }
    }
}
```

## Задание 2. Проверка корректности ввода пароля.

Разработана программа для проверки корректности пароля. Проверка осуществлялась по следующим критериям:

- длина от 8 до 16 символов;
- использование только латинских букв и цифр;
- наличие хотя бы одной заглавной буквы;
- наличие хотя бы одной цифры.

Для проверки применялось регулярное выражение, объединяющее все указанные условия.

```
import java.util.regex.*;

public class CheckPassword {
    public static void main(String[] args) throws
    ArrayIndexOutOfBoundsException {
        try {
            String text = "pAsSwOrD1337";
            Pattern pattern = Pattern.compile("^(?=.*[A-Z])(?=.*\\d)[A-Za-z\\d]{8,16}$");
            Matcher matcher = pattern.matcher(text);
            if (matcher.matches()) {
                System.out.println("Пароль корректен");
            }
            else {
                System.out.println("Пароль некорректен");
            }
        } catch (Exception e) {
            System.out.println("Произошла ошибка при проверке пароля: " +
            e.getMessage());
        }
    }
}
```

## **Задание 3. Поиск заглавной буквы после строчной**

Была написана программа, находящая в тексте случаи, когда сразу после строчной буквы следует заглавная без разделяющих символов. Такие фрагменты выделялись символами '!' с обеих сторон с использованием регулярных выражений и замены.

```
import java.util.regex.*;
public class Capital {
    public static void main(String[] args) {
        try {
            String text = "Бро здесьОшибка и тД";
            Pattern pattern = Pattern.compile("([а-яа-z])([А-ЯА-Z])");
            Matcher matcher = pattern.matcher(text);
            String result = matcher.replaceAll("$1!$2!");
            System.out.println(result);

        } catch (Exception e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
    }
}
```

#### Задание 4. Проверка корректности IP-адреса

Реализована программа для проверки корректности IPv4-адреса. Адрес считался корректным, если он состоял из четырёх чисел, разделённых точками, и каждое число находилось в диапазоне от 0 до 255. Проверка выполнялась с использованием регулярного выражения.

```
import java.util.regex.*;

public class CheckerIP {
    public static void main(String[] args) {
        try {
            String IP = "225.225.225.225";
            Pattern pattern = Pattern.compile("^((25[0-5]|2[0-4]\\d|1\\d{2}|[1-9]?\\d)\\.){3}" + "(25[0-5]|2[0-4]\\d|1\\d{2}|[1-9]?\\d)$");
            Matcher matcher = pattern.matcher(IP);
            if (matcher.matches()) {
                System.out.println("IP-адрес корректный!");
            }
            else {
                System.out.println("IP-адрес некорректный!");
            }
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

## Задание 5. Поиск слов, начинающихся с заданной буквы

Создана программа, выполняющая поиск всех слов в тексте, начинающихся с заданной пользователем буквы. Для поиска использовались границы слов и символные классы регулярных выражений.

```
import java.util.regex.*;

public class WordSearch {
    public static void main(String[] args) {
        try {
            String text = "public static void main String args";
            String letter = "S";
            Pattern pattern = Pattern.compile("(?i)(^|\\s)" + letter + "[a-zA-Z]*");
            Matcher matcher = pattern.matcher(text);
            boolean found = false;
            while (matcher.find()) {
                System.out.println(matcher.group());
                found = true;
            }
            if (!found) System.out.println("Таких слов нет.");
        } catch (Exception e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
    }
}
```

**Вывод:** в результате выполнения лабораторной работы были изучены особенности строк в Java и возможности регулярных выражений. Регулярные выражения показали свою эффективность при решении задач анализа и обработки текста. Полученные знания и навыки могут быть использованы при разработке прикладных программ, требующих валидации данных и обработки строковой информации.

<https://github.com/TRuslan666/ITiP/tree/main/лаб5>